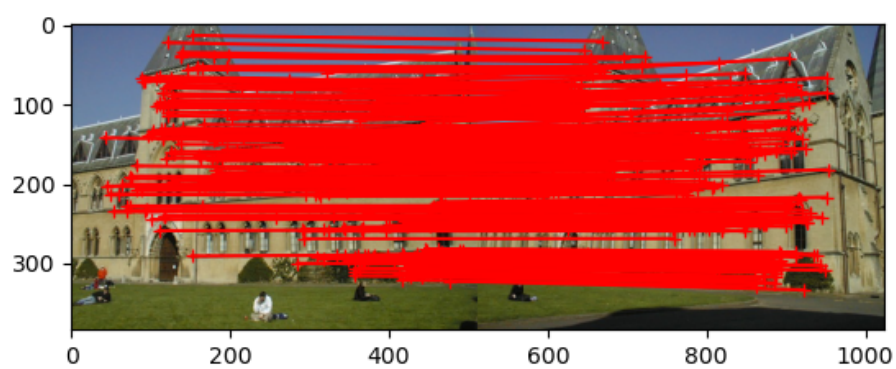


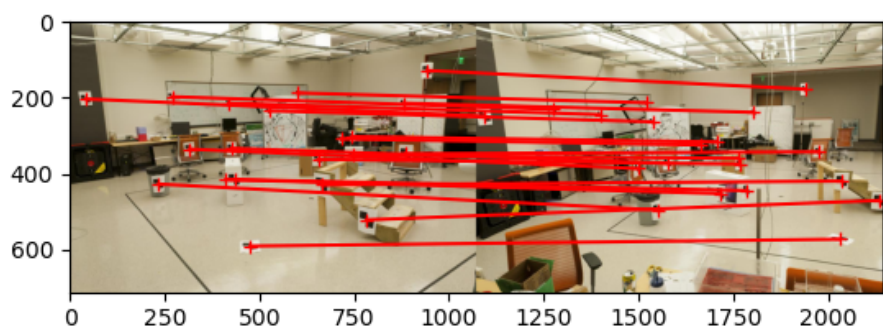
Homework 3

Haozheng Wang

November 2025



(a) library_matches



(b) lab_matches

Figure 1: matches visualization

1 Fundamental matrix estimation with ground truth matches

For normalized eight point algorithm, we first implement **normalize_points()** function so as to increase numerical stability. We need to center the set of points at the origin, and scale it so the mean squared distance between the origin and the point is 2 pixels. That is, given a set of points $(x_1, y_1), \dots, (x_n, y_n)$, if we denote $\bar{x} = \frac{1}{n} \sum_{i=1}^n x_i, \bar{y} = \frac{1}{n} \sum_{i=1}^n y_i, \sigma_x^2 = \frac{1}{n} \sum_{i=1}^n (x_i - \bar{x})^2, \sigma_y^2 = \frac{1}{n} \sum_{i=1}^n (y_i - \bar{y})^2$, then the point-wise transformation should look like:

$$(x_i, y_i) \longrightarrow \left(\frac{x_i - \bar{x}}{\sigma_x}, \frac{y_i - \bar{y}}{\sigma_y} \right), i = 1, \dots, n.$$

This transformation can be condensed into matrix form by using homogeneous coordinates:

$$\begin{bmatrix} x_1 & y_1 & 1 \\ x_2 & y_2 & 1 \\ \dots & \dots & \dots \\ x_n & y_n & 1 \end{bmatrix} \begin{bmatrix} \frac{1}{\sigma_x} & 0 & 0 \\ 0 & \frac{1}{\sigma_y} & 0 \\ -\frac{\bar{x}}{\sigma_x} & -\frac{\bar{y}}{\sigma_y} & 1 \end{bmatrix} = \begin{bmatrix} x_{1,\text{normalized}} & y_{1,\text{normalized}} & 1 \\ x_{2,\text{normalized}} & y_{2,\text{normalized}} & 1 \\ \dots & \dots & \dots \\ x_{n,\text{normalized}} & y_{n,\text{normalized}} & 1 \end{bmatrix}$$

i.e., $x^T T = x_{\text{normalized}}^T$, where $T = \begin{bmatrix} \frac{1}{\sigma_x} & 0 & 0 \\ 0 & \frac{1}{\sigma_y} & 0 \\ -\frac{\bar{x}}{\sigma_x} & -\frac{\bar{y}}{\sigma_y} & 1 \end{bmatrix}$, $x = (x, y, 1)^T$, $x_{\text{normalized}} = (x_{\text{normalized}}, y_{\text{normalized}}, 1)^T$.

To transform fundamental matrix back to original units, we derive the relation by writing down the epipolar constraint:

$$x_{\text{normalized}}'^T F_{\text{init}} x_{\text{normalized}} = x'^T T' F_{\text{init}} T^T x = 0.$$

That is, $F = T' F_{\text{init}} T^T$.

```

1 def normalize_points(pts):
2     # 1. calculate mean and std
3     mean = np.mean(pts, axis=0)
4     std = np.std(pts, axis=0)
5     # 2. build a transformation matrix
6     T = np.array([[1 / std[0], 0, 0],
7                  [0, 1 / std[1], 0],
8                  [-mean[0] / std[0], -mean[1] / std[1], 1]])
9     # 3. transform cartesian coordinates to homogeneous coordinates
10    homo_pts = np.ones((pts.shape[0], 3))
11    homo_pts[:, 0:2] = pts
12    # 4. center the set of points at the origin
13    # and scale it so [the mean squared distance between the origin
14    # and the point] is 2 pixels
15    normalized_pts = np.dot(homo_pts, T)[:, 0:2]
16    return normalized_pts, T

```

We then move on to implement the `fit_fundamental()` function to calculate the fundamental matrix from ground truth matches. For normalized eight point algorithm, we first normalize the two set of points using `normalize_points()` function and convert the coordinates to homogeneous coordinates. Then, we form matrix A in the same way as shown in the slides *07 Epipolar Geometry*. A singular value decomposition of matrix A follows to find the eigenvector of $A^T A$ with the smallest eigenvalue, i.e. flattened F_{init} . Finally, we enforce the rank-2 constraint on F_{init} by low-rank approximation based on SVD, and transform fundamental matrix back to original units. The unnormalized algorithm is completely a subset of the normalized one.

```

1 def fit_fundamental(matches):
2     n = matches.shape[0]
3     # 1. normalize the points and convert the coordinates to
4         homogeneous coordinates
5     pts_1 = matches[:, 0:2] # matches[:, :2] is a point in the
6         first image
7     pts_2 = matches[:, 2:4] # matches[:, 2:] is a corresponding
8         point in the second image
9     normalized_pts_1, T_1 = normalize_points(pts_1)
10    normalized_pts_2, T_2 = normalize_points(pts_2)
11    homo_pts_1 = np.ones((n, 3))
12    homo_pts_1[:, 0:2] = normalized_pts_1
13    homo_pts_2 = np.ones((n, 3))
14    homo_pts_2[:, 0:2] = normalized_pts_2
15    # 2. form matrix A
16    A = np.zeros((n, 9))
17    A[:, 0:3] = homo_pts_2[:, 0:1] * homo_pts_1
18    A[:, 3:6] = homo_pts_2[:, 1:2] * homo_pts_1
19    A[:, 6:9] = homo_pts_1
20    # 3. use SVD to find the eigenvector of ATA with the smallest
21        eigenvalue
22    U, S, V = np.linalg.svd(A)
23    F_init = np.reshape(V[np.argmax(S)], (3, 3))
24    # 4. enforce the rank-2 constraint
25    U_F, S_F, V_F = np.linalg.svd(F_init)
26    S_F_tilda = np.diag(S_F)
27    S_F_tilda[np.argmax(S_F)] = 0
28    F = np.dot(U_F, np.dot(S_F_tilda, V_F))
29    # 5. transform fundamental matrix back to original units
30    F = np.dot(T_2, np.dot(F, np.transpose(T_1)))
31    return F

```

The outcomes of normalized eight point algorithm are shown below:
For the **library** image pair, the fundamental matrix is:

$$\begin{bmatrix} -4.86210796e-08 & 9.98237781e-07 & -1.47341026e-04 \\ -5.80698246e-06 & -5.65060960e-08 & 1.07254893e-02 \\ 1.38164930e-03 & -9.63586245e-03 & -2.60674702e-01 \end{bmatrix}$$

and the residual is $4.857831302275997e-06$.

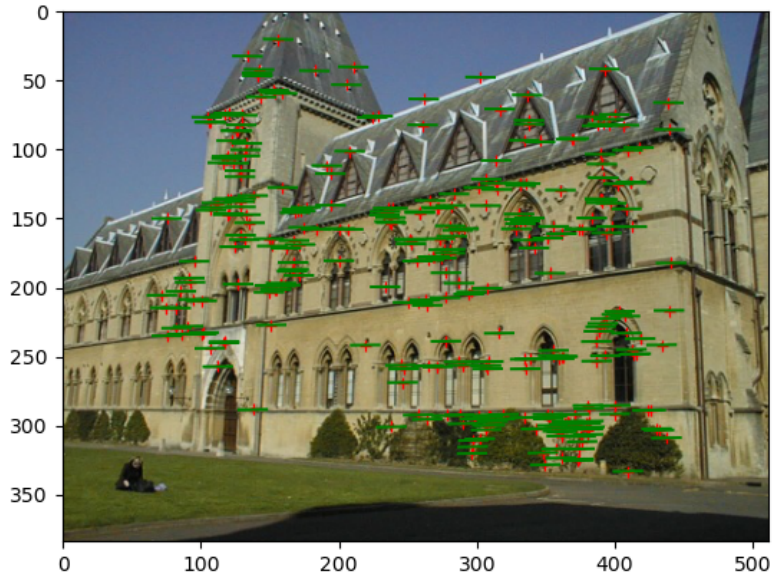


Figure 2: normalized library

For the **lab** image pair, the fundamental matrix is

$$\begin{bmatrix} -2.02478523e-07 & 2.78039315e-06 & -6.94781145e-04 \\ 1.92581510e-06 & -4.74398800e-07 & 5.59813726e-03 \\ -4.16157075e-05 & -7.69192568e-03 & 1.78588215e-01 \end{bmatrix}$$

and the residual is $2.2277388816999478e-05$.

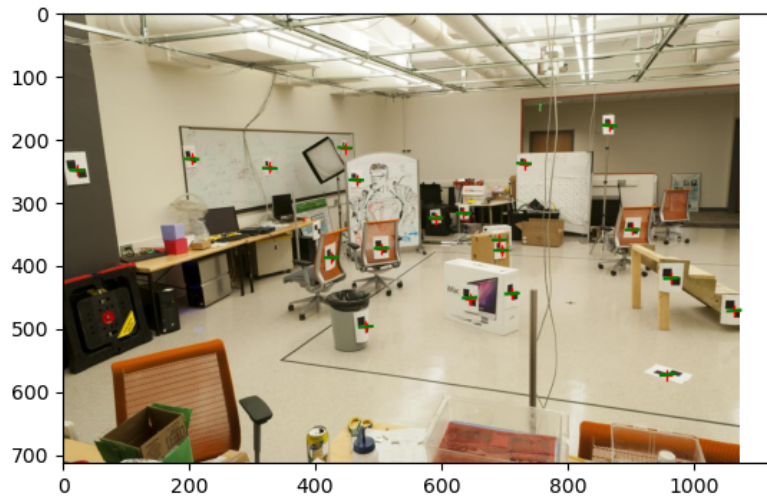


Figure 3: normalized lab

And the outcomes of unnormalized eight point algorithm are shown below:
For the **library** image pair, the fundamental matrix is:

$$\begin{bmatrix} -1.32341616e-06 & 1.36640519e-05 & -6.82803870e-04 \\ -2.88178174e-05 & 2.66440807e-07 & 4.09069255e-02 \\ 5.62362952e-03 & -3.72771609e-02 & -9.98451273e-01 \end{bmatrix}$$

and the residual is 0.00016149806538416032.

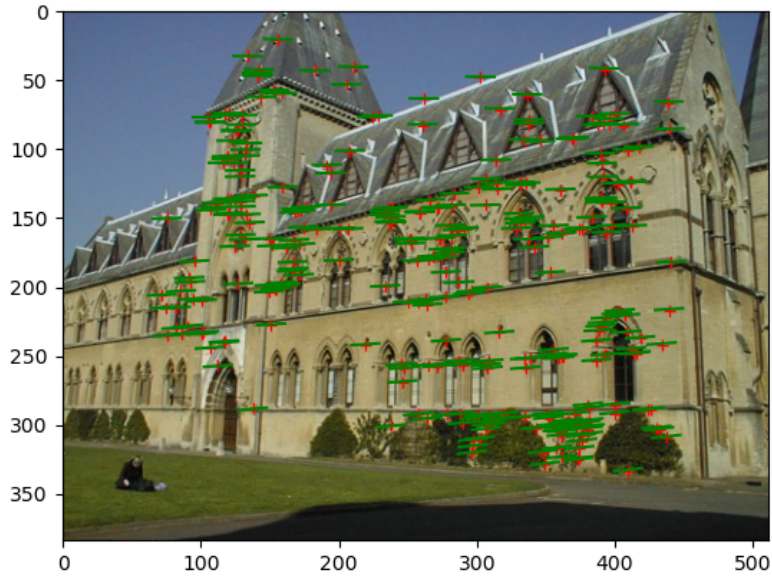


Figure 4: unnormalized library

For the **lab** image pair, the fundamental matrix is

$$\begin{bmatrix} -5.36264198e-07 & 7.90364771e-06 & -1.88600204e-03 \\ 8.83539184e-06 & 1.21321685e-06 & 1.72332901e-02 \\ -9.07382264e-04 & -2.64234650e-02 & 9.99500092e-01 \end{bmatrix}$$

and the residual is $2.2277388816999478e-05$.



Figure 5: unnormalized lab

It can be observed that the residual obtained from the fundamental matrix calculated using normalized point correspondences is significantly smaller than that obtained from unnormalized point correspondences. And if you observe and compare the visualized images carefully, you will notice that there are indeed differences in the effectiveness of the two methods, although it may seem subtle.

2 Camera calibration

As the procedure of constructing a system of equations for the camera calibration algorithm has been provided in detail, all we need is to simply repeat what we have implemented in the previous tasks: convert Cartesian coordinates to homogeneous coordinates, form matrix A, and use SVD to find the eigenvector of $A^T A$ with the smallest eigenvalue.

```

1 def calc_projection(points_2d, points_3d):
2     n = points_2d.shape[0]
3     # 1. convert the coordinates to homogeneous coordinates
4     homo_pts_2d = np.ones((n, 3))
5     homo_pts_2d[:, 0:2] = points_2d
6     homo_pts_3d = np.ones((n, 4))
7     homo_pts_3d[:, 0:3] = points_3d
8     # 2. form matrix A
9     A = np.zeros((2 * n, 12))
10    A[0:2*n:2, 0:4] = homo_pts_3d
11    A[1:2*n:2, 4:8] = homo_pts_3d
12    A[0:2*n:2, 8:12] = -homo_pts_2d[:, 0:1] * homo_pts_3d
13    A[1:2*n:2, 8:12] = -homo_pts_2d[:, 1:2] * homo_pts_3d
14    # 3. use SVD to find the eigenvector of ATA with the smallest
        eigenvalue
15    U, S, V = np.linalg.svd(A)
16    P = np.reshape(V[np.argmax(S)], (3, 4))
17    return P

```

For **lab_a**, the projection matrix we obtained is:

$$\begin{bmatrix} 3.09963996e-03 & 1.46204548e-04 & -4.48497465e-04 & -9.78930678e-01 \\ 3.07018252e-04 & 6.37193664e-04 & -2.77356178e-03 & -2.04144405e-01 \\ 1.67933533e-06 & 2.74767684e-06 & -6.83964827e-07 & -1.32882928e-03 \end{bmatrix}$$

with residual = 13.545832894885326 and distance = 3.965229591406653.

For **lab_b**, the projection matrix we obtained is:

$$\begin{bmatrix} 6.93154686e-03 & -4.01684470e-03 & -1.32602928e-03 & -8.26700554e-01 \\ 1.54768732e-03 & 1.02452760e-03 & -7.27440714e-03 & -5.62523256e-01 \\ 7.60946050e-06 & 3.70953989e-06 & -1.90203244e-06 & -3.38807712e-03 \end{bmatrix}$$

with residual = 13.545832894885326 and distance = 3.965229591406653.

It is noticed that residual is constantly larger than distance, which can be explained by the inequality $\sum_{i=1}^n \sqrt{x_{i,res}^2 + y_{i,res}^2} \geq \sqrt{\sum_{i=1}^n (x_{i,res}^2 + y_{i,res}^2)}$.

3 Calculate the camera matrices

The relationship between the projection matrix and the camera matrices is as follows:

$$P_{3 \times 4} = K[R|T] = [K_{3 \times 3} R_{3 \times 3} \quad K_{3 \times 3} T_{3 \times 1}]$$

where K is an upper triangular matrix and R is an orthonormal matrix.

We first perform RQ decomposition on left-most 3x3 matrix of $P_{3 \times 4}$ to get K and R. After obtaining these two matrices, T can be calculated by $K^{-1}P$. Finally, we normalize K to set $K[2,2]=1$, which accords with the form of the intrinsic matrix.

```

1 def rq_decomposition(P):
2     P_left = P[:, 0:3]
3     P_right = P[:, 3:4]
4     K, R = scipy.linalg.rq(P_left) # perform RQ decomposition on
5     # left-most 3x3 matrix of P(3 x 4) to get K, R
6     T = np.dot(np.linalg.inv(K), P_right) # calculate T by P = K[R|
7     # T]
8     K = K / K[2, 2] # normalize to set K[2, 2] = 1
9     return K, R, T

```

For **lab_a**,

$$K = \begin{bmatrix} 780.56750258 & 1.99846116 & 545.67044543 \\ 0. & 779.99109405 & 384.15959109 \\ 0. & 0. & 1. \end{bmatrix}$$

$$R = \begin{bmatrix} 0.84996668 & -0.5261548 & -0.02679126 \\ -0.13167568 & -0.16292373 & -0.97781254 \\ 0.51011583 & 0.83463584 & -0.20776153 \end{bmatrix}$$

$$T = \begin{bmatrix} -99.08341704 \\ 119.30063513 \\ -403.64591749 \end{bmatrix}$$

For **lab_b**,

$$K = \begin{bmatrix} 767.01522299 & 8.37481023 & 536.20619575 \\ 0. & 772.02253385 & 390.71267556 \\ 0. & 0. & 1. \end{bmatrix}$$

$$R = \begin{bmatrix} 0.43076697 & -0.90177034 & -0.03535629 \\ -0.21279825 & -0.06342282 & -0.97503561 \\ 0.8770158 & 0.42753689 & -0.21921561 \end{bmatrix}$$

$$T = \begin{bmatrix} 147.519785 \\ 113.64386256 \\ -390.48723313 \end{bmatrix}$$

For **library_a**,

$$K = \begin{bmatrix} -5.79790975e+02 & 1.11782147e-06 & 2.56991552e+02 \\ 0.00000000e+00 & -5.39711147e+02 & 2.04317558e+02 \\ 0.00000000e+00 & 0.00000000e+00 & 1.00000000e+00 \end{bmatrix}$$

$$R = \begin{bmatrix} -0.00966193 & -0.44135514 & -0.8972805 \\ -0.98017972 & -0.17338541 & 0.09583956 \\ -0.19787463 & 0.88042214 & -0.43093212 \end{bmatrix}$$

$$T = \begin{bmatrix} 6.48524008 \\ 1.71299092 \\ 28.03255592 \end{bmatrix}$$

For **library_b**,

$$K = \begin{bmatrix} -5.47469106e+02 & 7.30842577e-06 & 2.58430094e+02 \\ 0.00000000e+00 & -5.12933585e+02 & 2.04985542e+02 \\ 0.00000000e+00 & 0.00000000e+00 & 1.00000000e+00 \end{bmatrix}$$

$$R = \begin{bmatrix} 0.01861384 & -0.67624076 & -0.73644549 \\ -0.981381 & -0.15319028 & 0.11586227 \\ -0.19116708 & 0.72057697 & -0.6665013 \end{bmatrix}$$

$$T = \begin{bmatrix} 6.70661749 \\ 1.69482607 \\ 28.01539193 \end{bmatrix}$$

4 Triangulation

Given projections of a 3D point in two images with known camera matrices, we need to find the 3D Cartesian coordinates of the point. The idea for solving this problem is approximately the same as that for solving previous problems, and the key is still to construct the matrix A, i.e., to build the system of equations under homogeneous coordinates.

$$\begin{bmatrix} x_1 \\ y_1 \\ 1 \end{bmatrix} \cong \begin{bmatrix} p_{11}^{(1)} & p_{12}^{(1)} & p_{13}^{(1)} & p_{14}^{(1)} \\ p_{21}^{(1)} & p_{22}^{(1)} & p_{23}^{(1)} & p_{24}^{(1)} \\ p_{31}^{(1)} & p_{32}^{(1)} & p_{33}^{(1)} & p_{34}^{(1)} \end{bmatrix} \begin{bmatrix} X \\ Y \\ Z \\ W \end{bmatrix}$$

$$\begin{bmatrix} x_2 \\ y_2 \\ 1 \end{bmatrix} \cong \begin{bmatrix} p_{11}^{(2)} & p_{12}^{(2)} & p_{13}^{(2)} & p_{14}^{(2)} \\ p_{21}^{(2)} & p_{22}^{(2)} & p_{23}^{(2)} & p_{24}^{(2)} \\ p_{31}^{(2)} & p_{32}^{(2)} & p_{33}^{(2)} & p_{34}^{(2)} \end{bmatrix} \begin{bmatrix} X \\ Y \\ Z \\ W \end{bmatrix}$$

Take the equation corresponding to x_1 as an example:

$$x_1 = \frac{p_{11}^{(1)}X + p_{12}^{(1)}Y + p_{13}^{(1)}Z + p_{14}^{(1)}W}{p_{31}^{(1)}X + p_{32}^{(1)}Y + p_{33}^{(1)}Z + p_{34}^{(1)}W}$$

That is,

$$(p_{11}^{(1)} - x_1 p_{31}^{(1)})X + (p_{12}^{(1)} - x_1 p_{32}^{(1)})Y + (p_{13}^{(1)} - x_1 p_{33}^{(1)})Z + (p_{14}^{(1)} - x_1 p_{34}^{(1)})W = 0$$

By analogy, the other three equations corresponding to y_1, x_2, y_2 can be written with ease, and therefore we obtain the system of equations under homogeneous coordinates $A_{4 \times 4} X_{4 \times 1} = O_{4 \times 1}$:

$$\begin{bmatrix} p_{11}^{(1)} - x_1 p_{31}^{(1)} & p_{12}^{(1)} - x_1 p_{32}^{(1)} & p_{13}^{(1)} - x_1 p_{33}^{(1)} & p_{14}^{(1)} - x_1 p_{34}^{(1)} \\ p_{21}^{(1)} - y_1 p_{31}^{(1)} & p_{22}^{(1)} - y_1 p_{32}^{(1)} & p_{23}^{(1)} - y_1 p_{33}^{(1)} & p_{24}^{(1)} - y_1 p_{34}^{(1)} \\ p_{11}^{(2)} - x_2 p_{31}^{(2)} & p_{12}^{(2)} - x_2 p_{32}^{(2)} & p_{13}^{(2)} - x_2 p_{33}^{(2)} & p_{14}^{(2)} - x_2 p_{34}^{(2)} \\ p_{21}^{(2)} - y_2 p_{31}^{(2)} & p_{22}^{(2)} - y_2 p_{32}^{(2)} & p_{23}^{(2)} - y_2 p_{33}^{(2)} & p_{24}^{(2)} - y_2 p_{34}^{(2)} \end{bmatrix} \begin{bmatrix} X \\ Y \\ Z \\ W \end{bmatrix} = \begin{bmatrix} 0 \\ 0 \\ 0 \\ 0 \end{bmatrix}$$

What to do next is needless to say: compute the SVD decomposition of A , find the eigenvector of $A^T A$ with the smallest eigenvalue, and convert homogeneous coordinates back to Cartesian coordinates.

```

1 def triangulate_points(P1, P2, point1, point2):
2     A = np.zeros((4, 4))
3     A[0] = P1[0] - point1[0] * P1[2]
4     A[1] = P1[1] - point1[1] * P1[2]
5     A[2] = P2[0] - point2[0] * P2[2]
6     A[3] = P2[1] - point2[1] * P2[2]
7     U, S, V = np.linalg.svd(A)
8     homo_point_3d = V[np.argmax(S)]
9     point_3d = homo_point_3d[0:3] / homo_point_3d[3]
10    return point_3d

```

After computation, we obtain that for the **lab** pair,

$$\text{residual_a} = 10.899446059070128$$

$$\text{residual_b} = 1.5485148111134817$$

And for the **library** pair,

$$\text{residual_a} = 24.662071196868812$$

$$\text{residual_b} = 28.64953773526058$$

5 Fundamental matrix estimation without ground-truth matches

If there is no ground-truth matches, just create them yourself!

We choose to use SIFT operator to extract the features just as what we have done in the last assignment. However, when we carry out median truncation and some other techniques to select "good" matches after matching descriptors with Brute Force Matcher, it seems not to work well. As an improvement, we use **BFMatcher.knnMatch()** to search for two closest points in the second image to the point in the first image, and decide whether to retain this match by examining whether the gap between these two distances is large enough.

```
1 # compute SIFT keypoints and descriptors
2 sift = cv2.SIFT_create()
3 keypoints_1, descriptors_1 = sift.detectAndCompute(img_1, None)
4 keypoints_2, descriptors_2 = sift.detectAndCompute(img_2, None)
5 # match descriptors with Brute Force Matcher
6 matcher = cv2.BFMatcher()
7 matches = matcher.knnMatch(descriptors_1, descriptors_2, k=2)
8 # select good matches
9 good_matches = []
10 for match in matches:
11     if match[0].distance < 0.8 * match[1].distance:
12         good_matches.append(match[0])
```

About the implementation of RANSAC, it is almost the same as that in the last assignment. There are just two points to be noticed: Firstly, we calculate residuals as our geometric errors here, which are measured by $(x^T F x)^2$ for each pair of points. In addition, the inliers are recorded in preparation for display.

```
1 def align_pair(pts_1, pts_2):
2     n = len(pts_1)
3     homo_pts_1 = np.hstack((pts_1, np.ones((n, 1))))
4     homo_pts_2 = np.hstack((pts_2, np.ones((n, 1))))
5     iter_num = 1000
6     threshold = 1e-4
7     largest_inliers_num = 0
8     average_residual = 0
9     est_F = np.zeros((3, 3))
10    matches_inliers = []
11    for i in range(iter_num):
12        # randomly choose 8 samples, and fit the fundamental matrix
13        # F to those samples
14        random_sample_indices = np.random.choice(n, 8, replace=False)
15        random_pts_1 = [pts_1[j] for j in random_sample_indices]
16        random_pts_2 = [pts_2[j] for j in random_sample_indices]
17        matches = np.hstack((random_pts_1, random_pts_2))
18        F = fit_fundamental(matches)
19        # calculate residual and count the inliers
20        residual = np.diag(np.dot(np.dot(homo_pts_2, F), np.
21                                     transpose(homo_pts_1))) ** 2
```

```

20     inliers_num = np.sum([residual < threshold])
21     # choose the homo_matrix that has the largest set of
22     # inliers as out final est_homo
23     if inliers_num > largest_inliers_num:
24         largest_inliers_num = inliers_num
25         average_residual = np.mean(residual)
26         est_F = F
27     # prepare for displaying the inliers
28     inliers_1 = pts_1[residual < threshold]
29     inliers_2 = pts_2[residual < threshold]
30     matches_inliers = np.hstack((inliers_1, inliers_2))
31     return est_F, matches_inliers

```

For more code details, please refer to **main.py**.

For **house** image pair, we got 120 inliers in the experiment, and the average residual of the inliers is 2.9747538987655948e-05.

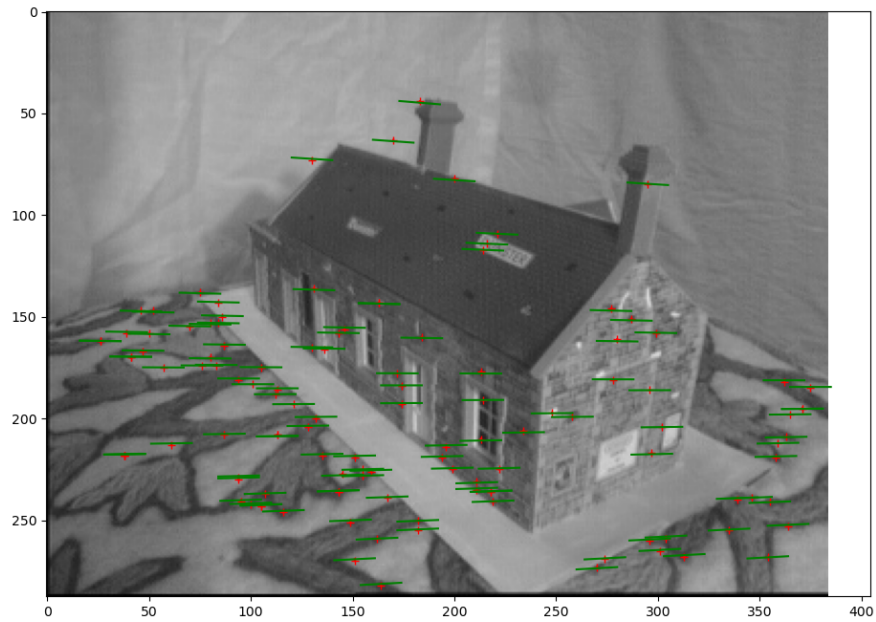


Figure 6: house_inliers

For **gaudi** image pair, we got 180 inliers in the experiment, and the average residual of the inliers is $2.5766979589835992\text{e-}05$.



Figure 7: gaudi_inliers